

# Chapter 6: Architectural Design

Software Engineering — Ian Sommerville (10th Edition)

Part 2 — Architectural Patterns

---

## Topics Covered

| # | Topic                            |
|---|----------------------------------|
| 1 | What are Architectural Patterns? |
| 2 | MVC (Model-View-Controller)      |
| 3 | Layered Architecture             |
| 4 | Repository Architecture          |
| 5 | Client-Server Architecture       |
| 6 | Pipe and Filter Architecture     |

---

## Topic 1: What are Architectural Patterns?

A pattern is a way of **representing, sharing, and reusing knowledge** about software systems — proven solutions to commonly occurring design problems.

**Every pattern must include:**

- **Pattern Name** — what it's called
- **Brief Description** — what it does
- **Graphical Model** — a diagram showing its structure
- **Example** — a real system where it has already been used
- **When to use** — conditions under which this pattern is suitable
- **Pros & Cons** — strengths and weaknesses

**The Fundamental Idea Behind All Patterns:**

Separation of Concern and Data Independence — each part handles ONE specific concern, and changes in one part don't ripple through everything else.

**Separation of Concern vs Patterns:**

- **Separation of Concern** = the GOAL
- **Architectural Patterns** = the TOOL to achieve that goal

**Key Takeaway:** Patterns are reusable, proven architectural solutions. They save time, reduce risk, and give teams a common vocabulary to discuss design.

---

## Topic 2: MVC (Model-View-Controller)

MVC splits an application into **3 separate components**, each with its own responsibility:

| Component  | Responsibility                                                           |
|------------|--------------------------------------------------------------------------|
| Model      | Handles data logic, accesses database, does authentication/update/delete |
| View       | Renders the data — what the user actually sees                           |
| Controller | The middleman — processes client requests and coordinates Model & View   |

### How it works — Step by Step:

1. User sends request → Controller
2. Controller processes request → sends to Model
3. Model handles data logic → accesses database
4. Model returns raw data → Controller
5. Controller passes data → View
6. View renders the data
7. View returns to Controller → Controller sends response to Client

Core idea: MVC is a way to **segregate data presentation from data logic**.

|               | Detail                                                                                             |
|---------------|----------------------------------------------------------------------------------------------------|
| When to use   | Multiple ways to view/interact with data; future presentation requirements unknown                 |
| Advantages    | Data changes independently of representation; supports multiple presentations of same data; adding |
| Disadvantages | Additional code complexity                                                                         |
| Real Examples | Django, Laravel, Ruby on Rails, ASP.NET                                                            |

**Key Takeaway:** MVC separates your application into 3 clean parts — Model (data), View (display), Controller (logic coordinator) — making it easier to maintain, update, and scale.

---

## Topic 3: Layered Architecture

Organizes system functionality into a **set of layers**, where each layer provides services to the layer above and relies on the layer below.

### Typical Layers (top to bottom):

- User Interface • Auth & Authorization • Business Logic • OS / Database

### The Rules:

- Upper layers talk to the layer **directly below** — no jumping allowed
- Lower layers **never call upper layers** — they only reply to inputs from above
- Requesting something means **iterating down through multiple layers**

|             | Detail                                                                                                 |
|-------------|--------------------------------------------------------------------------------------------------------|
| When to use | Building new facilities on existing systems; development spread across teams; multi-level security req |

|               |                                                                                                        |
|---------------|--------------------------------------------------------------------------------------------------------|
| Advantages    | Easy to embed NFRs (security) by inserting a layer; only adjacent layer affected on update; easy mult  |
| Disadvantages | Performance issue — request travels through multiple layers; difficult to code true independence; high |
| Real Example  | iLearn system — Utility → Application → Configuration → UI layers                                      |

**Connection to Part 1:** In Topic 10 (Security NFR) — 'Use a layered architecture with critical assets in inner layers' — that's exactly this pattern!

**Key Takeaway:** Layered architecture organizes a system into clean horizontal layers — each serving the one above. Great for security and maintainability, but can hurt performance due to multi-layer traversal.

## Topic 4: Repository Architecture

Sub-systems need to exchange data. There are two ways:

- **Way 1 (Repository):** All sub-systems access a single shared central database ✓
- **Way 2 (Direct):** Each sub-system maintains its own database and passes data explicitly ✗ (less efficient for large data)

When large amounts of data need to be shared, the **Repository model** is most commonly used.

**Important Distinction:** Repository is a **static structure** — it doesn't change at runtime. This differentiates it from Client-Server architecture (which is dynamic).

|               | Detail                                                                                                  |
|---------------|---------------------------------------------------------------------------------------------------------|
| When to use   | Large volumes of information stored long-term; data-driven systems where data inclusion triggers an a   |
| Advantages    | Components are independent — don't need to know about each other; changes propagate to all comp         |
| Disadvantages | Single point of failure — if repository goes down, whole system goes down; distributing repository acro |
| Real Examples | IDE (code editor, compiler, debugger share central repo); MIS systems                                   |

**Key Takeaway:** Repository centralizes all data in one place — making components independent and data management consistent. But the central repository is a single point of failure for the entire system.

## Topic 5: Client-Server Architecture

The **runtime organization of distributed systems** where servers provide specific services and clients access those services over a network.

| Component | Description                                            | Example                                      |
|-----------|--------------------------------------------------------|----------------------------------------------|
| Servers   | Provide specific services                              | Print server, File server, Management server |
| Clients   | Access services offered by servers; multiple instances | Browser, Email client, App                   |
| Network   | Connects clients and servers                           | Internet protocols                           |

Clients send requests via **remote procedure invocation** using **request-reply protocols**.

**Static vs Dynamic:**

|  | Repository | Client-Server |
|--|------------|---------------|
|--|------------|---------------|

|            |                                    |                                |
|------------|------------------------------------|--------------------------------|
| Structure  | Static — doesn't change at runtime | Dynamic — runtime organization |
| Focus      | Data sharing                       | Service sharing                |
| Components | Always the same                    | Clients join/leave dynamically |

|               | Detail                                                                                                 |
|---------------|--------------------------------------------------------------------------------------------------------|
| When to use   | Shared database accessed from range of locations; variable system load                                 |
| Advantages    | Resource sharing; independent servers; no single point of failure (overall); transparent upgradation   |
| Disadvantages | Each individual service is a single point of failure (DoS attacks); unpredictable performance (depends |
| Real Examples | Web applications, Email servers (Gmail), File sharing (Google Drive, Dropbox)                          |

**Key Takeaway:** Client-Server organizes a system into service providers (servers) and service consumers (clients) connected over a network. Great for distributed access, but individual servers can become points of failure.

## Topic 6: Pipe and Filter Architecture

Data flows through a **chain of processing components** where:

- **Filters** = processing components (each does ONE specific transformation on data)
- **Pipes** = connectors that pass data from one filter to the next

Flow: Input → [Filter 1] → pipe → [Filter 2] → pipe → [Filter 3] → Final Output

**Real World Examples:**

| Example              | Flow                                                                                        |
|----------------------|---------------------------------------------------------------------------------------------|
| Compiler             | Source Code → Lexical Analysis → Parsing → Semantic Analysis → Code Generation → Executable |
| Payment System       | Payment Request → Validate → Fraud Check → Process → Confirm → Notify                       |
| CNN Object Detection | Image → Convolution → Pooling → Flatten → Classification → Output Label                     |
| Unix Shell           | ls   grep '.txt'   sort   output                                                            |

|                 | Detail                                                                                                  |
|-----------------|---------------------------------------------------------------------------------------------------------|
| When to use     | Data processing applications (batch/transaction); compilers; embedded systems with limited user inter   |
| When NOT to use | Interactive/GUI systems — complex input streams (mouse clicks) are hard to convert to sequential str    |
| Advantages      | Easy to understand; supports transformation reuse; easy to evolve (add filters); can be sequential or c |
| Disadvantages   | Data format must be agreed upon between all filters; each filter must parse/unparse data; increases s   |

**Key Takeaway:** Pipe and Filter processes data through a sequence of independent transformation steps. Clean, reusable, and easy to extend — but requires all filters to agree on a common data format.

## Quick Comparison — All 5 Patterns

| Pattern | Core Idea                                | Best For                                   | Main Weakness                       |
|---------|------------------------------------------|--------------------------------------------|-------------------------------------|
| MVC     | Separate data, display, and control      | Web apps, UI-heavy systems                 | Additional code complexity          |
| Layered | Stack of layers, each serving the client | Enterprise systems, security-critical apps | Performance (multi-layer traversal) |

|               |                                             |                                             |                                                      |
|---------------|---------------------------------------------|---------------------------------------------|------------------------------------------------------|
| Repository    | All components share one central data store | Data-driven systems, IDEs                   | Single point of failure                              |
| Client-Server | Servers provide services, clients consume   | Distributed systems, web apps               | Individual server failure, unpredictable performance |
| Pipe & Filter | Data flows through chain of transformations | Compilers, batch processing, data pipelines | Complex data format overhead                         |

## Clarification: Layered vs Pipe and Filter

|             | Layered Architecture                           | Pipe and Filter                     |
|-------------|------------------------------------------------|-------------------------------------|
| What flows? | Requests/Calls between layers                  | Data between filters                |
| Purpose     | Organize system functionality/responsibilities | Process and transform data          |
| Components  | Layers (each provides services)                | Filters (each transforms data)      |
| Focus       | How system is STRUCTURED                       | What happens TO the data            |
| Example     | UI → Business Logic → Database                 | Raw Data → Filter → Filter → Output |

In one line: Layered = organizing the SYSTEM | Pipe & Filter = processing the DATA ■